Inheritance

2 Days Bootcamp

Rushikesh Phalke
(www.gainfins.com/rushikeshsp)

Department of Computer Engineering
Pimpri Chinchwad College of Engineering, Pune

PIMPRI CHINCHWAD EDUCATION TRUST
A Trusted Brand in Education Since 1990..

30/06/2018-01/07/2018

VISIONWARE
Pioneering Refining Developing

Introduction

- The capability of a class to derive properties and characteristics from another class is called Inheritance.
- Inheritance is one of the most important feature of Object Oriented Programming.

Sub Class / Derived Class: The class that inherits properties from another class is called Sub class or Derived Class.

Super Class / Base Class: The class whose properties are inherited by sub class is called Super Class or Base class.

VISIONWARE
Pioneering Refining Developing

Why & when to use Inheritance?

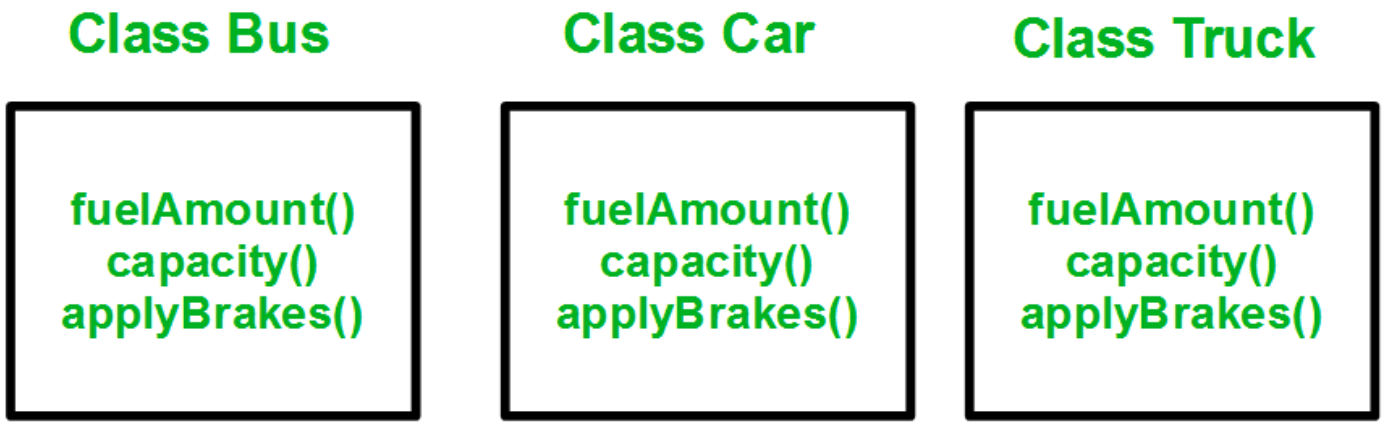
- Consider a group of vehicles. You need to create classes for Bus, Car and Truck.

- The methods fuelAmount(), capacity(), applyBrakes() will be same for all of the three classes.

- If we create these classes avoiding inheritance then we have to write all of these functions in each of the three classes.

VISIONWARE
Pioneering Refining Developing

Why & when to use Inheritance? Continued..

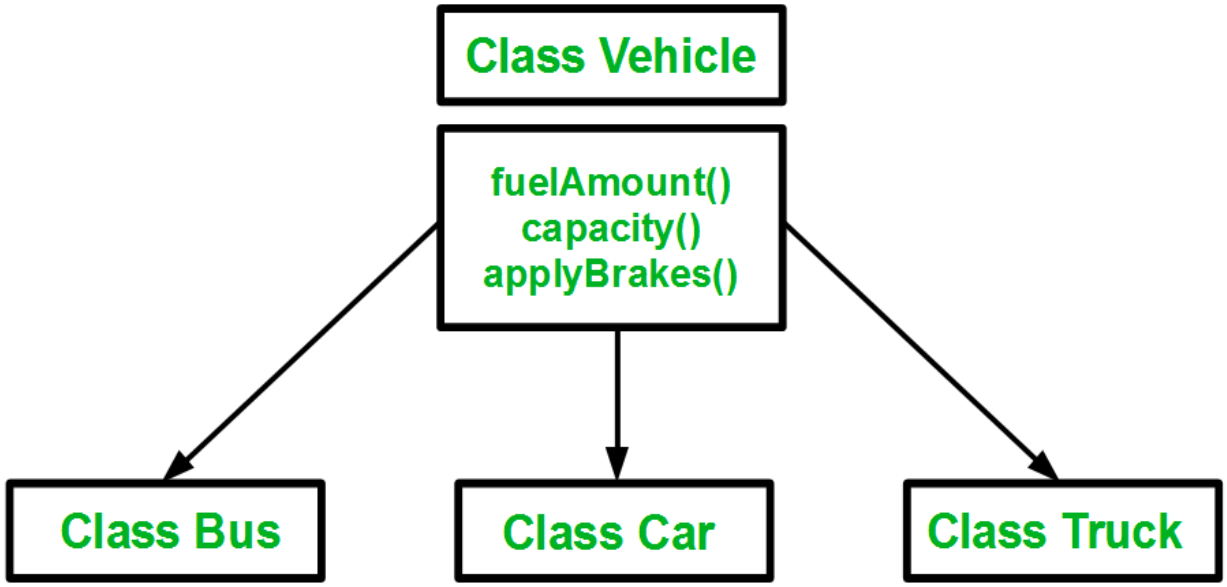


- We can clearly see that this process results in duplication of same code 3 times.
- This increases the chances of error and data redundancy.

VISIONWARE
Pioneering Refining Developing

Why & when to use Inheritance? Continued..

- To avoid this type of situation, inheritance is used.
- If we create a class Vehicle and write these three functions in it and inherit the rest of the classes from the vehicle class, then we can simply avoid the duplication of data and increase re-usability.



VISIONWARE
Pioneering Refining Developing

Implementing inheritance in C++

- For creating a sub-class which is inherited from the base class we have to follow the below syntax.
- Syntax :

```
class subclass_name : access_mode
base_class_name
{
  //body of subclass
};
```

VISIONWARE
Pioneering Refining Developing

C++ program to demonstrate implementation of Inheritance

```
#include <iostream>
using namespace std;
class Parent{
public:
    int id_p;
};
class Child : public Parent{
   public:
    int id_c;
};
int main(){
      Child obj1;
      obj1.id_c = 7;
      obj1.id_p = 91;
      cout << "Child id is " <<  obj1.id_c << endl;
      cout << "Parent id is " <<  obj1.id_p << endl;
      return 0;
 }
```

Base Class

access mode

Derived class

In this program the 'Child' class is publicly inherited from the 'Parent' class so the public data members of the class 'Parent' will also be inherited by the class 'Child'.

VISIONWARE
Pioneering Refining Developing

Modes of Inheritance

- **Public mode**: If we derive a sub class from a public base class. Then the public member of the base class will become public in the derived class and protected members of the base class will become protected in derived class.

- **Protected mode**: If we derive a sub class from a Protected base class. Then both public member and protected members of the base class will become protected in derived class.

Continued...

VISIONWARE
Pioneering Refining Developing

Modes of Inheritance continued..

- **Private mode**: If we derive a sub class from a Private base class. Then both public member and protected members of the base class will become Private in derived class.

- **Note** : The private members in the base class cannot be directly accessed in the derived class, while protected members can be directly accessed.

VISIONWARE
Pioneering Refining Developing

Example to show that a derived class doesn't inherit access to private data members.

```
class A {
    public:
        int x;
    protected:
        int y;
    private:
        int z;
};
```

```
class B : public A{
    // x is public
    // y is protected
    // z is not accessible from B
};
```

```
class C : protected A{
    // x is protected
    // y is protected
    // z is not accessible from C
};
```

```
class D : private A    // 'private' is default for classes
{
    // x is private
    // y is private
    // z is not accessible from D
};
```

VISIONWARE
Pioneering Refining Developing

The below table summarizes the above three modes and shows the access specifier of the members of base class in the sub class when derived in public, protected and private modes:

Base class member access specifier	Type of Inheritence		
	Public	Protected	Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Not accessible (Hidden)	Not accessible (Hidden)	Not accessible (Hidden)

VISIONWARE
Pioneering Refining Developing

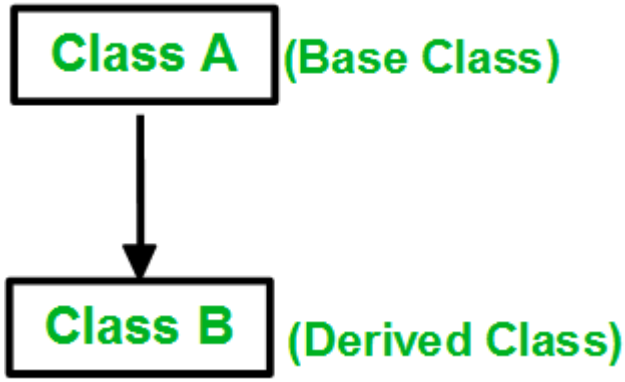
Types of Inheritance

- **Single Inheritance**
  - Class inherits from one base class
- **Multiple Inheritance**
  - Class inherits from multiple base classes
- **MultiLevel Inheritance**
  - Class inherits from inherited class
- **Hierarchical Inheritance**
  - One class inherited by more than one class
- **Hybrid Inheritance**
  - Combining more than one type of inheritance

VISIONWARE
Pioneering Refining Developing

Single Inheritance

- In single inheritance, a class is allowed to inherit from only one class. i.e. one sub class is inherited by one base class only.



- Syntax:

```
class subclass_name : access_mode base_class
{
    //body of subclass
};
```

VISIONWARE
Pioneering Refining Developing

Single Inheritance Program

```
#include <iostream>
using namespace std;
class Vehicle {
    public:
    Vehicle(){
    cout<< "This is a Vehicle"<< endl;
    }
};
class Car : public Vehicle {
public:
    Car(){
     cout << "This is a Car" << endl;
    }
 };
```

```
int main()
{
   // creating object of sub class will
   // invoke the constructor of base classes
   Car obj;
   return 0;
}
```

Guess the Output??

VISIONWARE
Pioneering Refining Developing

Single Inheritance Program

```
#include <iostream>
using namespace std;
class Vehicle {
    public:
    Vehicle(){
    cout<< "This is a Vehicle"<< endl;
    }
};
class Car : public Vehicle {
public:
    Car(){
     cout << "This is a Car" << endl;
    }
 };
```

```
int main()
{
   // creating object of sub class will
   // invoke the constructor of base classes
   Car obj;
   return 0;
}
```

Output:

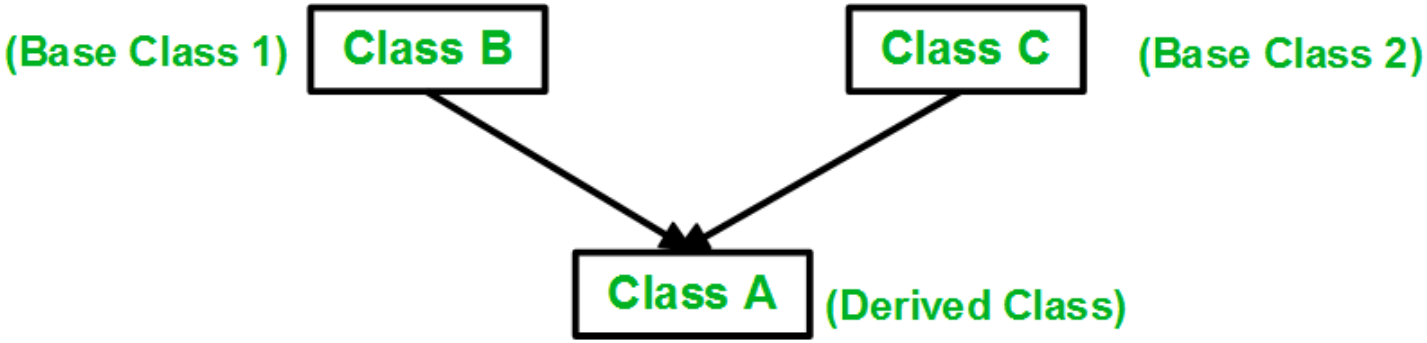
This is a vehicle

This is a Car

VISIONWARE
Pioneering Refining Developing

Multiple Inheritance

- Multiple Inheritance is a feature of C++ where a class can inherit from more than one classes. i.e one sub class is inherited from more than one base classes.



- Syntax:

```
class subclass_name : access_mode base_class1, access_mode base_class2, ....
{
    //body of subclass
};
```

- Here, the number of base classes will be separated by a comma (‘, ‘) and access mode for every base class must be specified.

VISIONWARE
Pioneering Refining Developing

program to explain multiple inheritance

```
#include <iostream>
using namespace std;

// first base class
class Vehicle {
  public:
    Vehicle()
    {
      cout << "This is a Vehicle" << endl;
    }
};

// second base class
class FourWheeler {
  public:
    FourWheeler()
    {
      cout << "This is a 4 wheeler Vehicle" << endl;
    }
};
```

```
// sub class derived from two base classes
class Car : public Vehicle, public
FourWheeler {


};
 // main function
int main() {
   // creating object of sub class will
   // invoke the constructor of base classes
   Car obj;
   return 0;
}
```

Guess the Output??

VISIONWARE
Pioneering Refining Developing

program to explain multiple inheritance

```
#include <iostream>
using namespace std;

// first base class
class Vehicle {
  public:
    Vehicle()
    {
      cout << "This is a Vehicle" << endl;
    }
};

// second base class
class FourWheeler {
  public:
    FourWheeler()
    {
      cout << "This is a 4 wheeler Vehicle" << endl;
    }
};
```

```
// sub class derived from two base classes
class Car : public Vehicle, public FourWheeler
{


};
 // main function
int main() {
    // creating object of sub class will
    // invoke the constructor of base classes
    Car obj;
    return 0;
}
```

Output:

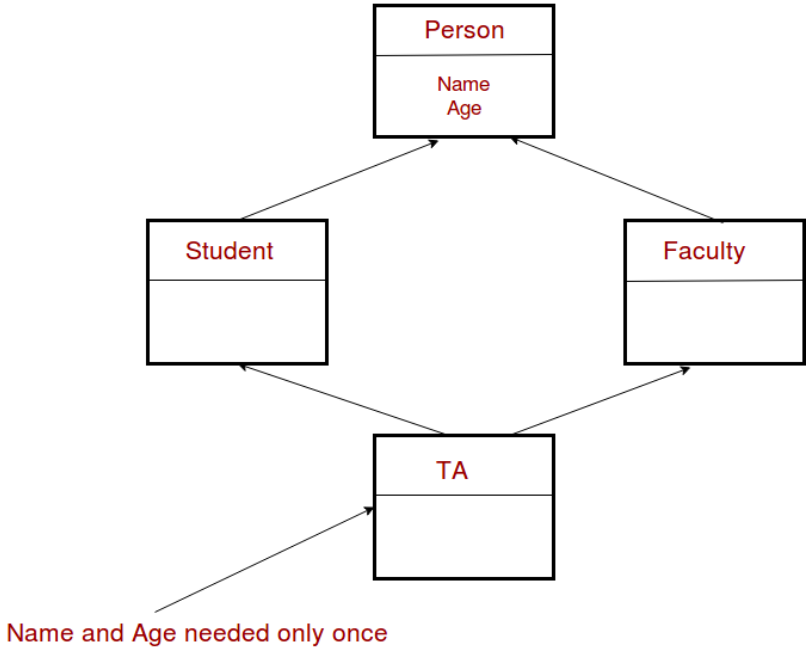
This is a Vehicle

This is a 4 wheeler Vehicle

VISIONWARE
Pioneering Refining Developing

The diamond problem

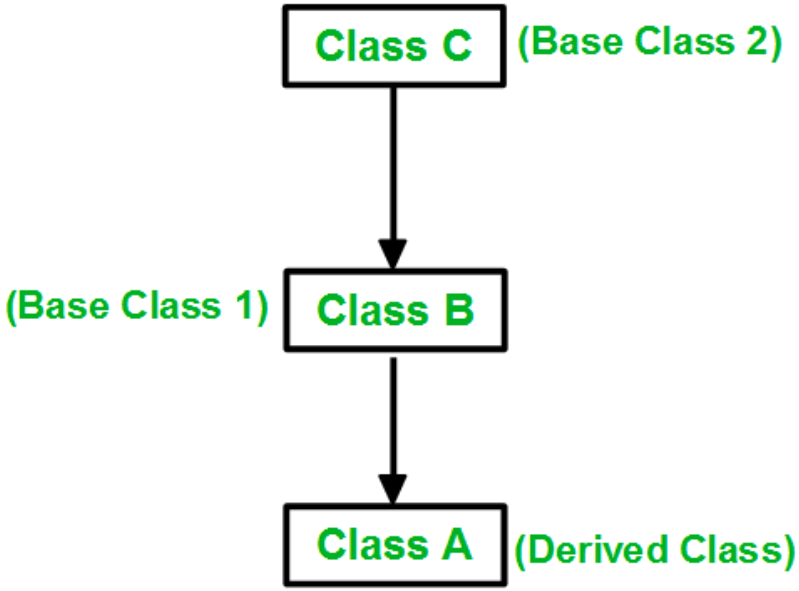
The diamond problem occurs when two superclasses of a class have a common base class. For example, in the following diagram, the TA class gets two copies of all attributes of Person class, this causes ambiguities.



VISIONWARE
Pioneering Refining Developing

Multilevel Inheritance

In this type of inheritance, a derived class is created from another derived class.



VISIONWARE
Pioneering Refining Developing

program to explain multilevel inheritance

```
#include <iostream>
using namespace std;

// base class
class Vehicle {
  public:
    Vehicle()
    {
      cout << "This is a Vehicle" << endl;
    }
};
class fourWheeler: public Vehicle{
  public:
    FourWheeler()
   {
      cout<<"Objects with 4 wheels are
vehicles"<<endl;
    }
};
```

```
// sub class derived from two base classes
class Car: public fourWheeler{
   public:
     car(){
       cout<<"Car has 4 Wheels"<<endl;
     }
};
int main(){
    //creating object of sub class will
    //invoke the constructor of base classes
    Car obj;
    return 0;
}
```

Guess the Output??

VISIONWARE
Pioneering Refining Developing

program to explain multilevel inheritance

```
#include <iostream>
using namespace std;

// base class
class Vehicle {
  public:
    Vehicle()
    {
      cout << "This is a Vehicle" << endl;
    }
};
class fourWheeler: public Vehicle{
  public:
    FourWheeler()
   {
      cout<<"Objects with 4 wheels are
vehicles"<<endl;
    }
};
```

```
// sub class derived from two base classes
class Car: public fourWheeler{
   public:
     car(){
       cout<<"Car has 4 Wheels"<<endl;
     }
};
int main(){
    //creating object of sub class will
    //invoke the constructor of base classes
    Car obj;
    return 0;
}
```

output:

This is a Vehicle

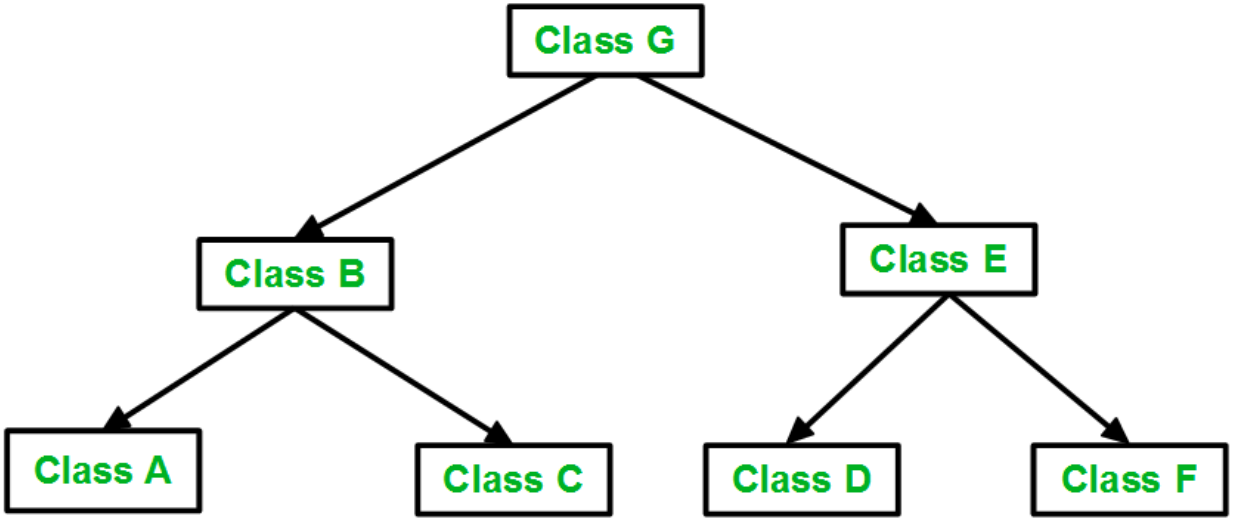
Objects with 4 wheels are vehicles

Car has 4 Wheels

VISIONWARE
Pioneering Refining Developing

Hierarchical Inheritance

In this type of inheritance, more than one sub class is inherited from a single base class. i.e. more than one derived class is created from a single base class.



VISIONWARE
Pioneering Refining Developing

program to explain hierarchical inheritance

```
#include <iostream>
using namespace std;

// base class
class Vehicle
{
  public:
    Vehicle()
    {
      cout << "This is a Vehicle" << endl;
    }
};
 // first sub class
class Car: public Vehicle
{

};
```

```
// second sub class
class Bus: public Vehicle
{

};
 // main function
int main()
{
    // creating object of sub class will
    // invoke the constructor of base
class
    Car obj1;
    Bus obj2;
    return 0;
}
```

Guess the Output??

VISIONWARE
Pioneering Refining Developing

program to explain hierarchical inheritance

```
#include <iostream>
using namespace std;

// base class
class Vehicle
{
  public:
    Vehicle()
    {
      cout << "This is a Vehicle" << endl;
    }
};
 // first sub class
class Car: public Vehicle
{

};
```

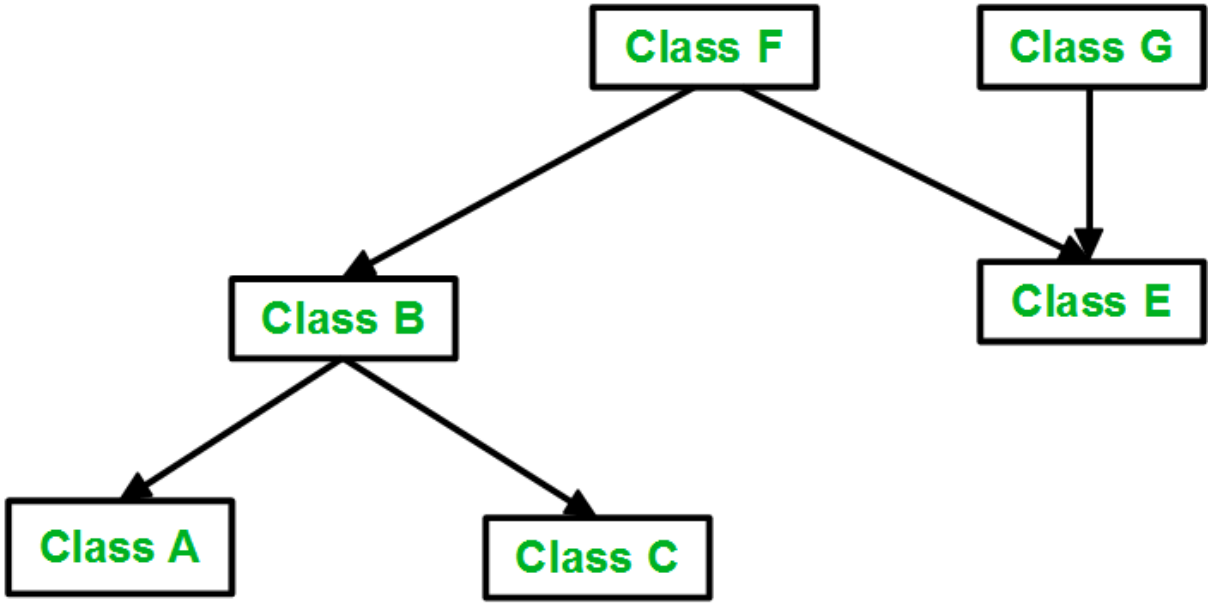
```
// second sub class
class Bus: public Vehicle
{

};
 // main function
int main()
{
    // creating object of sub class will
    // invoke the constructor of base class
    Car obj1;
    Bus obj2;
    return 0;
}
```

Output:
This is a Vehicle
This is a Vehicle

VISIONWARE
Pioneering Refining Developing

Hybrid Inheritance

Hybrid Inheritance is implemented by combining more than one type of inheritance. For example: Combining Hierarchical inheritance and Multiple Inheritance.



VISIONWARE
Pioneering Refining Developing

program to explain hybrid inheritance

```
#include <iostream>
using namespace std;
 // base class
class Vehicle {
  public:
    Vehicle()
    {
      cout << "This is a Vehicle" << endl;
    }
};
 //base class
class Fare {
    public:
    Fare()
    {
        cout<<"Fare of Vehicle\n";
    }
};
 // first sub class
class Car: public Vehicle
{
};
```

```
// second sub class
class Bus: public Vehicle, public Fare
{

};

// main function
int main()
{
    // creating object of sub class will
    // invoke the constructor of base class
    Bus obj2;
    return 0;
}
```

Guess the Output??

VISIONWARE
Pioneering Refining Developing

program to explain hybrid inheritance

```
#include <iostream>
using namespace std;

// base class
class Vehicle {
  public:
    Vehicle()
    {
      cout << "This is a Vehicle" << endl;
    }
};

//base class
class Fare {
    public:
    Fare()
    {
        cout<<"Fare of Vehicle\n";
    }
};

// first sub class
class Car: public Vehicle
{
};

// second sub class
class Bus: public Vehicle, public Fare
{

};

// main function
int main()
{
    // creating object of sub class will
    // invoke the constructor of base class
    Bus obj2;
    return 0;
}
```

Output:

This is a Vehicle
Fare of Vehicle

VISIONWARE
Pioneering Refining Developing